
Supervised and Self-Supervised Learning on Food-251

Anonymous Author(s)

Affiliation, Address

anon.email@example.org

Abstract

We train a residual convolutional network from scratch to classify the 251 fine-grained food categories of FoodX-251, under a hard budget of ten million trainable parameters and a single 8GB laptop GPU. We search among parameter-matched trunk variants using short, fixed-seed proxy runs on a held-out development split, then validate the selected trunk with a full training run reaching 61.57% top-1 / 85.67% top-5 accuracy before any recipe tuning. We describe a set of opt-in recipe extensions – stronger augmentation, Mixup/CutMix, an exponential moving average, a noise-robust loss – implemented and awaiting ablation, and outline a planned comparison against self-supervised pretraining under an equal-GPU-hour budget; both are reported where results exist and marked otherwise. Throughout, we find that measured wall-clock and memory behavior on commodity hardware – channel-width alignment, spatial resolution, and thermal throttling – shape the final design as much as raw parameter or FLOP counts do.

1 Introduction

FoodX-251 is a fine-grained food image classification benchmark spanning 251 categories, built from a large, web-crawled, noisily labeled training set with a smaller, manually curated and clean validation set [1]. Fine-grained food categories are hard to separate because inter-class visual differences (plating, preparation, garnish) can be subtler than the intra-class variation induced by lighting, angle, and portioning, and the label noise inherent to a web-crawled training set compounds the problem.

This work targets FoodX-251 under two constraints imposed by the setting rather than chosen for effect: the classifier is trained from scratch, with no ImageNet or other pretraining, under a budget of fewer than 10,000,000 trainable parameters, and every run happens on a single consumer laptop GPU with 8GB of VRAM. Neither constraint is negotiable, and together they shape everything that follows: an architecture search that screens on measured throughput and memory before ever measuring accuracy (Section 5.1), and a training recipe built to spend the wall-clock the 8GB budget buys, not just the parameter budget itself.

We make four contributions, in the order the underlying work was carried out: a measured, budget-constrained architecture search over residual-trunk variants (Section 5.1); a from-scratch training recipe modernized with mixed-precision training, a cosine schedule, and label smoothing, plus a set of opt-in extensions – stronger augmentation, Mixup/CutMix, an exponential moving average, a noise-robust loss – now implemented and awaiting ablation (Section 5.2); a full-length training run validating the selected trunk (Section 5.3); and a planned comparison of supervised training against self-supervised pretraining under an equal-GPU-hour budget (Section 5.4).

One design choice is worth flagging before the rest. Despite FoodX-251’s long-tailed reputation, the training split we measured is close to class-balanced (median 471 images per class; a single 34-image class drives the oft-cited 19.3x imbalance ratio), so class-balanced losses, logit adjustment, and long-tail-specific training are out of scope here – the real distributional problem is label noise, not long-tailedness (Section 3). More generally: the architecture search and one full training run

are complete at the time of writing; the recipe ablation and the self-supervised comparison are in progress, and are marked accordingly below rather than reported with numbers we have not measured.

2 Related work

We organize related work into three lanes, matching the three axes of this project: architecture, training recipe, and pretraining objective.

Parameter-efficient architecture design. Recent efficient-CNN work targets mobile and edge deployment under similar per-parameter and per-FLOP pressure: MobileNetV4 [2] and MobileOne [3] optimize explicitly for on-device latency rather than parameter count alone, and ConvNeXt V2 [4] revisits pure-convolutional design at a scale where masked-autoencoder pretraining is available – a regime this project’s from-scratch constraint excludes. None of these target fine-grained food classification, and our own literature search turned up no source addressing FoodX-251, Food-101, or Food-2K under a hard parameter budget (research/2026-07-30-architecture-brief.md) – the architecture decisions in Section 4 rest on general efficient-CNN principles and our own measurements rather than a food-specific prior.

Augmentation and label-noise robustness. RandAugment [5] and TrivialAugment [6] establish that automated or tuning-free augmentation policies match or beat searched policies at near-zero search cost; mixup [7] and CutMix [8] are complementary regularizers that mix inputs and labels rather than choosing among fixed operators. Generalized Cross Entropy [9] gives a noise-robust alternative to label-smoothed cross-entropy, directly relevant to FoodX-251’s web-crawled training labels. As with the architecture literature, none of these ablate sub-10M-parameter models trained for tens of epochs (research/2026-07-30-augmentation-noise-brief.md) – whether the accepted wisdom transfers to that regime is precisely what Section 5.2 is designed to check.

Self-supervised pretraining. SimCLR [10] established contrastive pretraining as a strong alternative to supervised training but relies on a large negative-sample batch to work well; at this project’s 8GB budget, reachable batch sizes fall below where SimCLR is known to degrade, which is why it is excluded here. BYOL [11], SimSiam [12], and Barlow Twins [13] instead avoid negative pairs entirely and remain viable at small batch sizes, making them the basis for the pretraining design in Section 5.4.

3 Dataset and protocol

FoodX-251 [1] contains 251 fine-grained food categories: a large, web-crawled training set with noisy labels, and a smaller, manually curated validation set of 11,994 images.

Class balance. Despite FoodX-251’s long-tailed reputation, per-class training counts are close to balanced: median 471 images per class, 10th/90th percentile 366/580, and exactly one class below 200 images (34) – that single outlier drives the frequently cited 19.3x imbalance ratio. We therefore do not use class-balanced losses, logit adjustment, or decoupled classifier retraining; the real distributional challenge is label noise, since training labels are web-crawled and validation labels are clean, not long-tailedness.

Validation split. Ablations and checkpoint selection must never touch the number this report ultimately quotes, or that number is implicitly selected-on. We split the 11,994-image validation set 50/50, stratified per class (every one of the 251 classes lands within one image of an exact half), into val-dev (6,063 images) and val-test (5,931 images), deterministically: src/make_val_split.py is a pure function of the official validation labels and a fixed seed (251), so the split is byte-identical on every regeneration and is not committed (splits/val_split.csv is gitignored and regenerated once per checkout). Every ablation and architecture comparison in this report reads val-dev; val-test is read exactly once, for the headline number in Section 5.3.

Proxy protocol. At the measured throughput of the selected trunk (about 1.2 minutes/epoch, Section 5.1), a 15-epoch run costs about 18 minutes, making comparisons that would be unaffordable at full length routine. Every architecture and recipe comparison below ranks candidates on a fixed-seed, 15-epoch val-dev proxy; the winning configuration is then confirmed with one full-length run.

88 **Logging.** Every run cited in this report is logged with a config hash, per-epoch train/validation
89 top-1 and top-5, wall-clock time, and peak VRAM (`src/runlog.py`), so every number below traces
90 back to a specific run.

91 4 Method

92 This section describes FoodCNN’s architecture, then the training recipe used to fit it.

93 4.1 Architecture

94 FoodCNN (`src/model.py`) is a plain residual CNN, sized to spend its parameter budget where it
95 can affect accuracy rather than where convention places it. A stem (3x3 stride-2 convolution, batch
96 normalization, ReLU, 3x3 stride-2 max-pool) takes the input to a quarter of its input resolution before
97 any residual stage runs. Four residual stages follow, each halving spatial resolution and doubling
98 channel width relative to the last – 64, 128, 256, then 512 channels, with block counts [2, 2, 2, 1] –
99 so that the final stage, carrying one block rather than two, is what keeps the trunk under budget.

100 Each residual block places batch normalization after both convolutions in the main branch and,
101 whenever the shortcut is not the identity, after the 1x1 shortcut projection as well – an unnormalized
102 branch should not be added to a normalized one. The second batch norm’s scale (γ) in every block is
103 zero-initialized, so every block begins training as an identity map; this is what makes a high learning
104 rate safe under warmup. The nonlinearity is applied after the residual addition, not before it, so each
105 block is an actual nonlinearity rather than an affine detour.

106 **A genuinely global head.** Pooling to (1, 1) rather than the (7, 7) map convolutions naturally leave
107 behind matters: at (7, 7), flattening produces a $512 \times 7 \times 7 = 25,088$ -feature vector, and a single
108 subsequent linear layer to 251 classes costs 6.4M parameters – 94% of the entire model, spent on
109 its least expressive layer. Pooling to (1, 1) first reduces that same layer to `Linear(512, 251)`, 129k
110 parameters.

111 **Parameter budget.** With the fixes above, FoodCNN totals 6,576,955 trainable parameters – 65.8%
112 of the 10M budget – with roughly 98% of them inside the convolutional trunk rather than the
113 classifier head.

114 **A rejected alternative.** A depthwise-separable version of the same trunk (MobileNetV2-style
115 inverted residuals, retained in `src/model.py` as a documented negative result rather than deleted) was
116 measured head-to-head with the plain-convolution trunk on this hardware: 2.55M parameters, 621
117 img/s, 3.83 GiB peak memory, against the plain-convolution trunk’s 6.58M parameters, 918 img/s,
118 1.28 GiB peak (full detail in Section C). Depthwise convolutions here cost activation memory and
119 wall-clock far more than they save in parameter count, and under this project’s constraints, throughput
120 and memory – not parameter count – are what actually bind (Section 6).

121 4.2 Training recipe

122 The reference training loop is modernized in ways that cost nothing at this parameter budget:
123 `torch.autocast("cuda", dtype=torch.bfloat16)` without a `GradScaler` (bf16 does not need
124 dynamic loss scaling), `channels_last` memory format on both model and inputs, a cosine learning-
125 rate schedule with five-epoch linear warmup in place of step decay, `CrossEntropyLoss` with label
126 smoothing 0.1, SGD with Nesterov momentum, eight dataloader workers (up from four), a 176px
127 training crop (down from 224), and no `DataParallel` wrapper – there is only one GPU to shard
128 across, and the wrapper’s scatter/gather adds a per-step cost and a `module.` prefix to every checkpoint
129 key for no benefit here.

130 4.3 Recipe extensions

131 Four further axes are implemented as opt-in flags to the training script, all defaulting to off
132 so that nothing about the recipe above changes unless explicitly requested: an augmentation
133 switch between no augmentation, `TrivialAugment` [6], and `RandAugment` [5]; batch-wise Mixup
134 [7] and CutMix [8], scored against the pre-mix hard label since a mixed batch has no single

Table 1: 15-epoch val-dev proxy results for four candidate trunks, all under the 10M-parameter cap.

Variant	Params	val-dev top-1	val-dev top-5	Peak VRAM
[2,2,2,2] 64-448	9.46M	49.99%	78.29%	2.04 GiB
[2,2,4,1] 64-512	8.94M	49.73%	77.93%	2.14 GiB
baseline [2,2,2,1] 64-512	6.58M	48.82%	77.12%	1.99 GiB
5-stage 48-512 [2,2,2,2,1]	9.08M	47.42%	75.94%	1.61 GiB

correct class to evaluate top-1/top-5 against; an exponential moving average of model weights (`torch.optim.swa_utils.AveragedModel`, batch-norm running statistics left live rather than averaged); and Generalized Cross Entropy [9] as an alternative to label-smoothed cross-entropy for the training set’s label noise. Section 5.2 reports which, if any, improve on the recipe above.

5 Experiments

Accuracy figures below are val-dev top-1/top-5 unless stated otherwise; val-test is read exactly once, in Section 5.3, per the protocol in Section 3. All measurements were taken on the project GPU (RTX 5050 Laptop, 8GB VRAM) in its performance power profile; Section 6 explains why that qualifier matters for every throughput number below.

5.1 Architecture search

A first pass screened trunk variants by synthetic throughput and memory alone (benchmarks/bench.py, benchmarks/trunk_variants.py, full sweep in Section B), before spending any GPU-hours on accuracy. Two findings from that pass decided which variants were worth an accuracy proxy at all. First, stage widths must be multiples of 32: a 72-144-288-576 trunk has **fewer** parameters than the five-stage candidate below yet runs 46% slower, falling off the tensor-core fast path. Second, final spatial resolution dominates cost more than any other single knob: dropping the stem’s max-pool to preserve a 6x6 final feature map, at identical parameter count, costs 2.9x throughput (1960 to 684 img/s) and doubles peak memory.

Four candidates under the parameter cap were then measured for accuracy with fixed-seed, 15-epoch val-dev proxies (benchmarks/proxy_sweep.py):

The five-stage variant’s extra downsample to a 3x3 final map costs 1.4-2.6 accuracy points despite being the second-largest candidate by parameter count, so the throughput and memory advantage it showed in the synthetic pass does not transfer to a real training signal; it is ruled out. The remaining three sit within 1.2 points of each other on a single seed each – not enough to call a winner with confidence. We adopt the baseline [2,2,2,1] 64-512 trunk provisionally: it is statistically indistinguishable from the other two here, uses 30% fewer parameters than the nominal leader, and that headroom has not measured as worth anything yet – better spent, for now, on the recipe tuning in Section 5.2. [2,2,4,1] and [2,2,2,2] 64-448 remain defined in benchmarks/trunk_variants.VARIANTS and can be re-proxied without repeating the measurement work above, if the tuned baseline underperforms expectations.

5.2 Recipe ablation

TODO — Not yet run. Planned: fixed-seed, 15-epoch val-dev proxies on the selected trunk, sweeping (i) learning rate in {0.05, 0.1, 0.2, 0.4} at batch 256; (ii) TrivialAugment vs. RandAugment; (iii) Mixup/CutMix on/off; (iv) EMA on/off; (v) batch size in {160, 256, 512} with learning rate scaled linearly – throughput is flat across this range at fixed resolution (1738-1785 img/s, batch 160-768), so this axis is a gradient-noise and batch-norm-statistics question, not a throughput one; (vi) cross-entropy with label smoothing vs. Generalized Cross Entropy, adopted only if it wins. Results and the resulting recipe decision replace this paragraph once measured.

180 5.3 Full supervised training run

181 A full 90-epoch run on the unmodified baseline trunk, with the training recipe above and none of
182 Section 5.2’s extensions, validated that the pipeline survives full length before spending further GPU-
183 hours tuning it. It converged to **61.57% val-dev top-1 / 85.67% top-5** (best checkpoint at epoch
184 82: 62.08% / 85.52%), rising from about 7% at epoch 0 and comfortably past both a from-scratch
185 Food-101 anchor (mid-50s%) and this trunk’s own 15-epoch proxy (Table 1, 47-50%). No crashes,
186 NaNs, or out-of-memory errors occurred across the full run; peak memory was 1.99 GiB, still far
187 under the 8 GiB ceiling at full length. Wall-clock was 2.31 hours, 28% above the approximately 1.8
188 hour synthetic estimate from the architecture search’s throughput sweep – per-batch time rose from
189 about 0.16s early in the run to about 0.26s past epoch 60 as sustained load heat-soaked the laptop
190 GPU (Section 6). This run validates the pipeline; it is not the project’s final supervised number, which
191 is what Section 5.2’s tuned recipe produces, read once against val-test with FixRes-style test-time
192 resolution correction [14] (train at 176px, evaluate at 224px center crop).

193 **TODO — val-test headline number: pending the recipe decision in Section 5.2.**

194 5.4 Self-supervised comparison

195 **TODO — Not started. Planned: SimSiam/BYOL pretraining for 200 epochs at 176px (about**
196 **8 GPU-hours), finetuned at 176px (about 1.8 GPU-hours), compared against an equal-GPU-**
197 **hour supervised control (about 8 GPU-hours) trained on the same trunk – without that control,**
198 **the comparison cannot separate “self-supervised pretraining helped” from “this trunk keeps**
199 **improving with more epochs regardless of objective.” SimCLR [10] is excluded: it is known**
200 **to degrade below roughly 1,000-example batches, unreachable within an 8GB budget. Results**
201 **replace this paragraph once measured.**

202 6 Discussion

203 **Hardware measurement mattered as much as the architecture.** Several findings above only
204 exist because throughput and memory were measured rather than assumed. Stage widths must be
205 multiples of 32 to stay on the tensor-core fast path (a 46% throughput gap at equal parameter count,
206 Section 5.1); final spatial resolution costs more than almost any other knob (2.9x throughput, 2x
207 memory, for an unchanged parameter count); the dataloader is never the bottleneck at this resolution
208 – plain crop-and-flip sustains 4.4-4.7k images/second at any worker count against roughly 1.7-1.9k
209 images/second of GPU demand, and even TrivialAugment keeps 2.5x headroom at eight workers,
210 falling to 1.4x at sixteen as oversubscription costs throughput on this machine’s 15GB of system
211 RAM; and batch size is throughput-flat from 160 to 768 (1738-1785 img/s) while memory scales
212 linearly from 1.28 to 5.64 GiB, meaning the GPU is compute-saturated at the smallest batch size
213 tried, not idle.

214 **Thermal throttling makes a short benchmark an upper bound, not an estimate.** The GPU
215 runs against a roughly 50W power cap with its clock oscillating 2175-2340 MHz against a 3090 MHz
216 maximum, and an early version of the throughput benchmark swung 45% on one variant between
217 sweeps before being changed to time three 40-step rounds and keep the best (sweeps now agree within
218 2-5%, and rankings are stable). Even so, the full run in Section 5.3 took 28% longer than its own
219 synthetic estimate: sustained 90-epoch load heat-soaks this laptop GPU in a way no short benchmark
220 window reproduces. Every throughput figure in this report is an upper bound on real epoch time and
221 a ratio between variants, not a wall-clock promise.

222 **Limitations.** The architecture comparison in Section 5.1 rests on a single 15-epoch seed per
223 variant; the 1.2-point spread separating three of the four candidates is well within plausible run-to-
224 run noise, which is why the baseline trunk is adopted provisionally rather than declared a winner
225 outright. All measurements are specific to one machine (RTX 5050 Laptop, 8GB VRAM) and need
226 not transfer, even in ratio, to other hardware. Training is from scratch throughout, by the project’s
227 own constraint, so absolute accuracy figures here are not directly comparable to pretrained food-
228 classification baselines without accounting for that.

229 7 Conclusion

Under a 10M-parameter budget and a single 8GB laptop GPU, a plain residual CNN with a genuinely global pooling head reaches 61.57% val-dev top-1 / 85.67% top-5 on FoodX-251 in a full 90-epoch run, before any recipe tuning. Reaching that number meant treating measured wall-clock and memory behavior – channel-width alignment, spatial resolution, thermal throttling – as design constraints alongside the parameter cap itself: a fifth residual stage and a depthwise-separable trunk both looked attractive on paper and measurably lost once actually run. The recipe ablation (Section 5.2) and the self-supervised comparison against an equal-compute supervised control (Section 5.4) remain open.

TODO — Revisit this conclusion once Section 5.2 and Section 5.4 have real results.

References

- [1] P. Kaur, K. Sikka, W. Wang, S. Belongie, and A. Divakaran, “FoodX-251: A Dataset for Fine-grained Food Classification,” *arXiv preprint arXiv:1907.06167*, 2019, [Online]. Available: <https://arxiv.org/abs/1907.06167>
- [2] D. Qin *et al.*, “MobileNetV4 - Universal Models for the Mobile Ecosystem,” *arXiv preprint arXiv:2404.10518*, 2024, [Online]. Available: <https://arxiv.org/abs/2404.10518>
- [3] P. K. A. Vasu, J. Gabriel, J. Zhu, O. Tuzel, and A. Ranjan, “MobileOne: An Improved One millisecond Mobile Backbone,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 7907–7917. [Online]. Available: <https://arxiv.org/abs/2206.04040>
- [4] S. Woo *et al.*, “ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 16133–16142. [Online]. Available: <https://arxiv.org/abs/2301.00808>
- [5] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, “RandAugment: Practical automated data augmentation with a reduced search space,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2020, pp. 702–703. [Online]. Available: <https://arxiv.org/abs/1909.13719>
- [6] S. G. Müller and F. Hutter, “TrivialAugment: Tuning-free Yet State-of-the-Art Data Augmentation,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021. [Online]. Available: <https://arxiv.org/abs/2103.10158>
- [7] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “mixup: Beyond Empirical Risk Minimization,” in *International Conference on Learning Representations*, 2018. [Online]. Available: <https://arxiv.org/abs/1710.09412>
- [8] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, “CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019. [Online]. Available: <https://arxiv.org/abs/1905.04899>
- [9] Z. Zhang and M. Sabuncu, “Generalized Cross Entropy Loss for Training Deep Neural Networks with Noisy Labels,” in *Advances in Neural Information Processing Systems*, 2018. [Online]. Available: <https://arxiv.org/abs/1805.07836>
- [10] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A Simple Framework for Contrastive Learning of Visual Representations,” in *Proceedings of the 37th International Conference on Machine Learning*, 2020, pp. 1597–1607. [Online]. Available: <https://arxiv.org/abs/2002.05709>
- [11] J.-B. Grill *et al.*, “Bootstrap Your Own Latent - A New Approach to Self-Supervised Learning,” in *Advances in Neural Information Processing Systems*, 2020, pp. 21271–21284. [Online]. Available: <https://arxiv.org/abs/2006.07733>
- [12] X. Chen and K. He, “Exploring Simple Siamese Representation Learning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021. [Online]. Available: <https://arxiv.org/abs/2011.10566>
- [13] J. Zbontar, L. Jing, I. Misra, Y. LeCun, and S. Deny, “Barlow Twins: Self-Supervised Learning via Redundancy Reduction,” in *Proceedings of the 38th International Conference on Machine Learning*, 2021, pp. 12310–12320. [Online]. Available: <https://arxiv.org/abs/2103.03230>
- [14] H. Touvron, A. Vedaldi, M. Douze, and H. Jégou, “Fixing the train-test resolution discrepancy,” *Advances in Neural Information Processing Systems*, vol. 32, 2019, [Online]. Available: <https://arxiv.org/abs/1906.06423>

Table 2: Synthetic throughput/memory sweep over trunk variants under the 10M-parameter cap, 176px training resolution, batch 160. “90 ep” is the projected wall-clock for a full 90-epoch run at the measured throughput; “Final map” is the trunk’s spatial resolution immediately before global average pooling.

Variant	Params	img/s	90 ep	Peak VRAM	Final map
baseline [2,2,2,1] 64-512	6.58M	1688	1.8 h	1.26 GiB	6x6
[2,2,3,1] 64-512	7.76M	1585	1.9 h	1.31 GiB	6x6
[2,2,4,1] 64-512	8.94M	1475	2.0 h	1.36 GiB	6x6
[2,3,3,1] 64-512	8.06M	1460	2.0 h	1.39 GiB	6x6
[2,2,2,2] 64-448	9.46M	1608	1.8 h	1.30 GiB	6x6
wider@11 64-320-512	7.98M	1494	2.0 h	1.30 GiB	6x6
5 stages 48-512 [2,2,2,2,1]	9.08M	1960	1.5 h	1.04 GiB	3x3
uniformly wider 72-576	8.31M	1062	2.8 h	1.41 GiB	6x6
5 stages, no stem max-pool	9.08M	684	4.3 h	2.16 GiB	6x6

Table 3: Depthwise-separable vs. plain-convolution trunk, measured head-to-head on the project GPU (176px, batch 160, bf16, channels_last).

Trunk	Params	img/s	Peak VRAM
Depthwise-separable (MobileNetV2-style)	2.55M	621	3.83 GiB
Plain-convolution residual (adopted)	6.58M	918	1.28 GiB

A Technical appendices and supplementary material

Supplementary tables referenced from Section 4 and Section 5.1 follow: the full synthetic throughput sweep behind the architecture search, and additional detail on the rejected depthwise-separable trunk.

B Full architecture-screening sweep

Table 1 in the main text reports 15-epoch accuracy proxies for four trunk variants. Those four were selected from a larger synthetic screening pass over throughput, parameter count, and memory alone (benchmarks/bench.py, benchmarks/trunk_variants.py), all at 176px training resolution and batch 160, before any accuracy was measured. The full sweep:

The last row isolates the resolution effect cited in Section 6: identical parameter count to the row above it, but with the stem’s max-pool removed to keep the final feature map at 6x6 instead of 3x3, at a cost of 2.9x throughput and roughly 2x peak memory. The “uniformly wider 72-576” row isolates the tensor-core alignment effect: fewer parameters than the five-stage variant, at 46% lower throughput, purely from using channel widths that are not multiples of 32.

C The rejected depthwise-separable trunk

Before the residual trunk in Section 4, a MobileNetV2-style depthwise-separable version of the same network was implemented and measured head-to-head on the same hardware. It is kept in src/model.py, unused by FoodCNN.features, as a documented negative result rather than deleted:

Fewer parameters did not mean faster or lighter here: depthwise convolutions trade parameter count for activation memory and wall-clock, and under this project’s binding constraints – throughput and

319 memory, not raw parameter count – that trade loses.¹ The plain-convolution trunk was adopted on
320 this basis before the architecture search in Section 5.1 ever began.

¹This comparison’s 918 img/s baseline figure is lower than the 1688 img/s recorded for the same trunk in Table 2. The two were measured in separate benchmarking passes – this comparison predates `benchmarks/bench.py`’s current three-round, keep-the-best methodology – and, per the thermal-throttling behavior discussed in Section 6, absolute throughput on this hardware is not stable across passes. The qualitative conclusion, that the depthwise-separable trunk trades parameters for memory and wall-clock, does not depend on which figure is used.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [NA].
- [NA] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for NA).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While “[Yes]” is generally preferable to “[No]”, it is perfectly acceptable to answer “[No]” provided a proper justification is given (e.g., “error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering “[No]” or “[NA]” is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”.
- Keep the checklist subsection headings, questions/answers and guidelines below.
- Do not modify the questions and only use the provided macros for your answers.

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer NA means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

366 Question: Does the paper discuss the limitations of the work performed by the authors?

367 Answer: **[TODO]**

368 Justification: **[TODO]**

369 Guidelines:

- 370 • The answer NA means that the paper has no limitation while the answer No means that the
371 paper has limitations, but those are not discussed in the paper.
- 372 • The authors are encouraged to create a separate “Limitations” section in their paper.
- 373 • The paper should point out any strong assumptions and how robust the results are to
374 violations of these assumptions (e.g., independence assumptions, noiseless settings, model
375 well-specification, asymptotic approximations only holding locally). The authors should
376 reflect on how these assumptions might be violated in practice and what the implications
377 would be.
- 378 • The authors should reflect on the scope of the claims made, e.g., if the approach was only
379 tested on a few datasets or with a few runs. In general, empirical results often depend on
380 implicit assumptions, which should be articulated.
- 381 • The authors should reflect on the factors that influence the performance of the approach. For
382 example, a facial recognition algorithm may perform poorly when image resolution is low
383 or images are taken in low lighting. Or a speech-to-text system might not be used reliably
384 to provide closed captions for online lectures because it fails to handle technical jargon.
- 385 • The authors should discuss the computational efficiency of the proposed algorithms and
386 how they scale with dataset size.
- 387 • If applicable, the authors should discuss possible limitations of their approach to address
388 problems of privacy and fairness.
- 389 • While the authors might fear that complete honesty about limitations might be used by
390 reviewers as grounds for rejection, a worse outcome might be that reviewers discover limita-
391 tions that aren’t acknowledged in the paper. The authors should use their best judgment and
392 recognize that individual actions in favor of transparency play an important role in devel-
393 oping norms that preserve the integrity of the community. Reviewers will be specifically
394 instructed to not penalize honesty concerning limitations.

395 3. Theory Assumptions and Proofs

396 Question: For each theoretical result, does the paper provide the full set of assumptions and a
397 complete (and correct) proof?

398 Answer: **[TODO]**

399 Justification: **[TODO]**

400 Guidelines:

- 401 • The answer NA means that the paper does not include theoretical results.
- 402 • All the theorems, formulas, and proofs in the paper should be numbered and cross-
403 referenced.
- 404 • All assumptions should be clearly stated or referenced in the statement of any theorems.
- 405 • The proofs can either appear in the main paper or the supplemental material, but if they
406 appear in the supplemental material, the authors are encouraged to provide a short proof
407 sketch to provide intuition.
- 408 • Inversely, any informal proof provided in the core of the paper should be complemented by
409 formal proofs provided in appendix or supplemental material.
- 410 • Theorems and Lemmas that the proof relies upon should be properly referenced.

411 4. Experimental Result Reproducibility

412 Question: Does the paper fully disclose all the information needed to reproduce the main exper-
413 imental results of the paper to the extent that it affects the main claims and/or conclusions of the
414 paper (regardless of whether the code and data are provided or not)?

415 Answer: **[TODO]**

416 Justification: **[TODO]**

417 Guidelines:

- 418 • The answer NA means that the paper does not include experiments.
- 419 • If the paper includes experiments, a No answer to this question will not be perceived well by
420 the reviewers: Making the paper reproducible is important, regardless of whether the code
421 and data are provided or not.
- 422 • If the contribution is a dataset and/or model, the authors should describe the steps taken to
423 make their results reproducible or verifiable.
- 424 • Depending on the contribution, reproducibility can be accomplished in various ways. For
425 example, if the contribution is a novel architecture, describing the architecture fully might
426 suffice, or if the contribution is a specific model and empirical evaluation, it may be
427 necessary to either make it possible for others to replicate the model with the same dataset,
428 or provide access to the model. In general, releasing code and data is often one good way
429 to accomplish this, but reproducibility can also be provided via detailed instructions for
430 how to replicate the results, access to a hosted model (e.g., in the case of a large language
431 model), releasing of a model checkpoint, or other means that are appropriate to the research
432 performed.
- 433 • While NeurIPS does not require releasing code, the conference does require all submissions
434 to provide some reasonable avenue for reproducibility, which may depend on the nature of
435 the contribution. For example
 - 436 (a) If the contribution is primarily a new algorithm, the paper should make it clear how to
437 reproduce that algorithm.
 - 438 (b) If the contribution is primarily a new model architecture, the paper should describe the
439 architecture clearly and fully.
 - 440 (c) If the contribution is a new model (e.g., a large language model), then there should either
441 be a way to access this model for reproducing the results or a way to reproduce the model
442 (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - 443 (d) We recognize that reproducibility may be tricky in some cases, in which case authors
444 are welcome to describe the particular way they provide for reproducibility. In the case
445 of closed-source models, it may be that access to the model is limited in some way (e.g.,
446 to registered users), but it should be possible for other researchers to have some path to
447 reproducing or verifying the results.

448 5. Open Access to Data and Code

449 Question: Does the paper provide open access to the data and code, with sufficient instructions
450 to faithfully reproduce the main experimental results, as described in supplemental material?

451 Answer: **[TODO]**

452 Justification: **[TODO]**

453 Guidelines:

- 454 • The answer NA means that paper does not include experiments requiring code.
- 455 • Please see the NeurIPS code and data submission guidelines ([https://neurips.cc/
456 public/guides/CodeSubmissionPolicy](https://neurips.cc/public/guides/CodeSubmissionPolicy)) for more details.
- 457 • While we encourage the release of code and data, we understand that this might not be
458 possible, so “No” is an acceptable answer. Papers cannot be rejected simply for not including
459 code, unless this is central to the contribution (e.g., for a new open-source benchmark).

- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental Setting/Details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment Statistical Significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer “Yes” if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).

- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments Compute Resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of Ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader Impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point

550 out that an improvement in the quality of generative models could be used to generate
 551 deepfakes for disinformation. On the other hand, it is not needed to point out that a generic
 552 algorithm for optimizing neural networks could enable people to train models that generate
 553 Deepfakes faster.

- 554 • The authors should consider possible harms that could arise when the technology is being
 555 used as intended and functioning correctly, harms that could arise when the technology is
 556 being used as intended but gives incorrect results, and harms following from (intentional or
 557 unintentional) misuse of the technology.
- 558 • If there are negative societal impacts, the authors could also discuss possible mitigation
 559 strategies (e.g., gated release of models, providing defenses in addition to attacks, mecha-
 560 nisms for monitoring misuse, mechanisms to monitor how a system learns from feedback
 561 over time, improving the efficiency and accessibility of ML).

562 11. Safeguards

563 Question: Does the paper describe safeguards that have been put in place for responsible release
 564 of data or models that have a high risk for misuse (e.g., pre-trained language models, image
 565 generators, or scraped datasets)?

566 Answer: **[TODO]**

567 Justification: **[TODO]**

568 Guidelines:

- 569 • The answer NA means that the paper poses no such risks.
- 570 • Released models that have a high risk for misuse or dual-use should be released with
 571 necessary safeguards to allow for controlled use of the model, for example by requiring that
 572 users adhere to usage guidelines or restrictions to access the model or implementing safety
 573 filters.
- 574 • Datasets that have been scraped from the Internet could pose safety risks. The authors should
 575 describe how they avoided releasing unsafe images.
- 576 • We recognize that providing effective safeguards is challenging, and many papers do not
 577 require this, but we encourage authors to take this into account and make a best faith effort.

578 12. Licenses for Existing Assets

579 Question: Are the creators or original owners of assets (e.g., code, data, models), used in the
 580 paper, properly credited and are the license and terms of use explicitly mentioned and properly
 581 respected?

582 Answer: **[TODO]**

583 Justification: **[TODO]**

584 Guidelines:

- 585 • The answer NA means that the paper does not use existing assets.
- 586 • The authors should cite the original paper that produced the code package or dataset.
- 587 • The authors should state which version of the asset is used and, if possible, include a URL.
- 588 • The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- 589 • For scraped data from a particular source (e.g., website), the copyright and terms of service
 590 of that source should be provided.
- 591 • If assets are released, the license, copyright information, and terms of use in the package
 592 should be provided. For popular datasets, <https://paperswithcode.com/datasets> has
 593 curated licenses for some datasets. Their licensing guide can help determine the license of
 594 a dataset.
- 595 • For existing datasets that are re-packaged, both the original license and the license of the
 596 derived asset (if it has changed) should be provided.

597 • If this information is not available online, the authors are encouraged to reach out to the
598 asset’s creators.

599 13. New Assets

600 Question: Are new assets introduced in the paper well documented and is the documentation
601 provided alongside the assets?

602 Answer: **[TODO]**

603 Justification: **[TODO]**

604 Guidelines:

- 605 • The answer NA means that the paper does not release new assets.
- 606 • Researchers should communicate the details of the dataset/code/model as part of their
607 submissions via structured templates. This includes details about training, license, limita-
608 tions, etc.
- 609 • The paper should discuss whether and how consent was obtained from people whose asset
610 is used.
- 611 • At submission time, remember to anonymize your assets (if applicable). You can either
612 create an anonymized URL or include an anonymized zip file.

613 14. Crowdsourcing and Research with Human Subjects

614 Question: For crowdsourcing experiments and research with human subjects, does the paper
615 include the full text of instructions given to participants and screenshots, if applicable, as well
616 as details about compensation (if any)?

617 Answer: **[TODO]**

618 Justification: **[TODO]**

619 Guidelines:

- 620 • The answer NA means that the paper does not involve crowdsourcing nor research with
621 human subjects.
- 622 • Including this information in the supplemental material is fine, but if the main contribution
623 of the paper involves human subjects, then as much detail as possible should be included in
624 the main paper.
- 625 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or
626 other labor should be paid at least the minimum wage in the country of the data collector.

627 15. Institutional Review Board (IRB) Approvals or Equivalent for Research with Human 628 Subjects

629 Question: Does the paper describe potential risks incurred by study participants, whether such
630 risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals
631 (or an equivalent approval/review based on the requirements of your country or institution) were
632 obtained?

633 Answer: **[TODO]**

634 Justification: **[TODO]**

635 Guidelines:

- 636 • The answer NA means that the paper does not involve crowdsourcing nor research with
637 human subjects.
- 638 • Depending on the country in which research is conducted, IRB approval (or equivalent)
639 may be required for any human subjects research. If you obtained IRB approval, you should
640 clearly state this in the paper.

- 641 • We recognize that the procedures for this may vary significantly between institutions and
642 locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines
643 for their institution.
- 644 • For initial submissions, do not include any information that would break anonymity (if
645 applicable), such as the institution conducting the review.

646 16. Declaration of LLM Usage

647 Question: Does the paper describe the usage of LLMs if it is an important, original, or non-
648 standard component of the core methods in this research? Note that if the LLM is used only for
649 writing, editing, or formatting purposes and does *not* impact the core methodology, scientific
650 rigor, or originality of the research, declaration is not required.

651 Answer: **[TODO]**

652 Justification: **[TODO]**

653 Guidelines:

- 654 • The answer NA means that the core method development in this research does not involve
655 LLMs as any important, original, or non-standard components.
- 656 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not be
657 described.